The common reverse-proxy description assumes that your application is running in a separate process from the reverse proxy itself. That is the case if you're running on Linux and was the default for IIS up until ASP.NET Core 3.0.

In ASP.NET Core 3.0, ASP.NET Core switched to using an in-process hosting model by default for applications deployed to IIS. In this model, IIS hosts your application directly inside the IIS process, reducing interprocess communication and boosting performance.

You can switch to the out-of-process hosting model with IIS if you wish, which can sometimes be useful for troubleshooting problems. Rick Strahl has an excellent post on the differences between the hosting models, how to switch between them, and the advantages of each: "ASP.NET Core In Process Hosting on IIS with ASP.NET Core" at http://mng.bz/QmEv.

In general, you shouldn't need to worry about the differences between the hosting models, but it's something to be aware of if you're deploying to IIS. If you choose to use the out-of-process hosting model, you should use the `UseIISIntegration()` extension method. If you use the in-process model, use `UseIIS()`. Alternatively, play it safe and use both; the correct extension method is activated based on the hosting model used in production. Neither extension does anything if you don't use IIS.

When running behind IIS, these extension methods configure your app to pair with IIS so that it can seamlessly accept requests. Among other things, the extensions do the following:

- Define the URL that IIS uses to forward requests to your app and configures your app to listen on this URL
- Configure your app to interpret requests coming from IIS as coming from the client by setting up header forwarding
- Enable Windows authentication if required

Adding the IIS extension methods is the only change you need to make to your application to host in IIS (and even then, only when using a custom application builder). But there's one additional aspect to be aware of when you publish your app. As with legacy .NET Framework ASP.NET, IIS relies on a web.config file to configure the applications it runs. It's important that